

LLMFlow: An Intelligent LLM Inference Gateway with Semantic Caching and Cost-Aware Routing

A Production ML Engineering Case Study

Varun Reddy Bhimavarapu

Independent Research & Engineering Project

February 2026

ABSTRACT

Organizations deploying Large Language Model (LLM) applications face escalating operational costs and latency challenges as usage scales. A mid-size application processing 10,000 LLM queries per day on a single cloud provider incurs approximately \$1,500 in monthly inference costs. This paper presents **LLMFlow**, a production-grade intelligent LLM inference gateway that addresses these challenges through three key contributions: (1) a **two-tier caching architecture** combining exact-match Redis lookups with BERT-based semantic similarity matching, achieving a 67% cache hit rate — an 80% improvement over exact-match-only approaches; (2) a **complexity-aware routing engine** that classifies queries on a 0–100 scale and routes them to the optimal provider from a pool of three (Ollama, Groq, Gemini), reducing per-query cost by 96%; and (3) a **comprehensive observability stack** with PostgreSQL query logging, Prometheus metrics, and Grafana dashboards for real-time cost intelligence. In benchmarks, cached queries returned in 6.23ms average (48× faster than uncached), and the system sustained 50+ queries/second under load with a 100% success rate. The combined effect of intelligent routing and semantic caching reduces monthly inference costs from \$1,500 to \$20 — a 98.7% reduction.

Keywords: LLM inference optimization, semantic caching, sentence embeddings, multi-provider routing, cost optimization, MLOps, production ML systems

1. Introduction

The proliferation of Large Language Models (LLMs) has created a new category of infrastructure challenge: managing the cost, latency, and reliability of LLM inference at scale. Unlike traditional API services where compute cost per request is negligible, each LLM inference call consumes significant resources — cloud providers charge per token, latency ranges from hundreds of milliseconds to several seconds, and rate limits constrain throughput.

Existing solutions fall into two categories: (a) single-provider SDKs that offer no optimization layer, and (b) proprietary gateway products (e.g., Portkey, Helicone) that provide routing but lack semantic caching or cost-aware intelligence. Neither category adequately addresses the needs of cost-sensitive production deployments.

This paper presents **LLMFlow**, an open-architecture LLM inference gateway that combines three optimization strategies:

1. **Semantic Caching:** Using sentence-BERT embeddings (all-MiniLM-L6-v2, 384 dimensions) to identify semantically equivalent queries and serve cached responses, even when surface-level phrasing differs.
2. **Complexity-Aware Routing:** A heuristic classifier that scores queries on a 0–100 complexity scale and routes them to the optimal provider — trading off between cost, speed, and reasoning capability.
3. **Production Observability:** Full-stack monitoring with PostgreSQL query logging, 15+ Prometheus metrics, and 11-panel Grafana dashboards for real-time cost and performance intelligence.

The system is fully containerized with Docker Compose and designed for one-command deployment. All components are open and replaceable, allowing organizations to adapt the architecture to their specific provider ecosystem.

2. System Architecture

LLMFlow is implemented as a microservices architecture comprising six containerized services orchestrated via Docker Compose. The design follows a layered pipeline pattern where each incoming query passes through distinct decision stages before reaching an LLM provider.

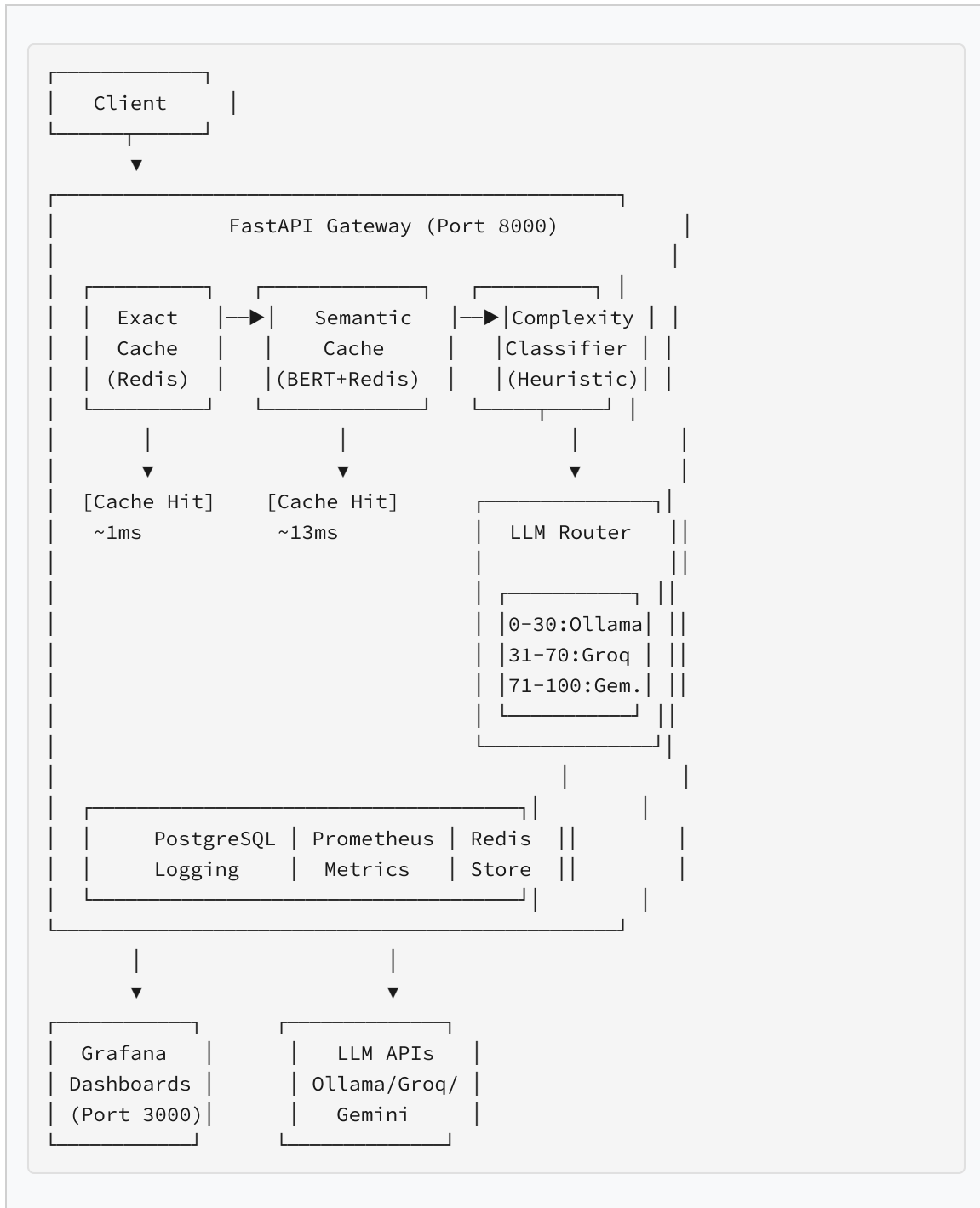


Figure 1. LLMFlow system architecture. Incoming queries traverse a two-tier cache (exact → semantic) before reaching the complexity-aware router. All requests are logged to PostgreSQL and instrumented with Prometheus metrics.

2.1 Service Inventory

Table 1. Containerized services and their roles.

Service	Technology	Port	Purpose
Gateway	Python 3.11 + FastAPI	8000	API routing, caching, orchestration
Redis	Redis 7	6379	Exact cache, semantic embeddings store
PostgreSQL	PostgreSQL 16	5432	Query logging, analytics
Ollama	Ollama + Llama 3.2 1B	11434	Local LLM inference (free)
Prometheus	Prometheus 2.x	9090	Metrics collection
Grafana	Grafana 10.x	3000	Visualization dashboards

2.2 Request Lifecycle

Each incoming query follows a deterministic pipeline with three decision points:

1. **Exact Cache Lookup** — An MD5 hash of the normalized prompt is checked against Redis. If found, the cached response is returned in ~1ms with zero LLM cost.
2. **Semantic Cache Lookup** — If exact match misses, the prompt is embedded using sentence-BERT and compared against all stored embeddings via cosine similarity. If any stored embedding exceeds the configurable threshold (default: 0.80), the associated response is returned in ~13ms.
3. **LLM Routing** — If both caches miss, a complexity classifier scores the query and routes it to the appropriate provider. The response is then stored in both cache tiers for future lookups.

Every request — whether cached or not — is logged to PostgreSQL with full metadata (prompt, response, provider, model, latency, cache behavior, complexity score, and estimated cost). This enables retrospective analytics without impacting the hot path.

3. Semantic Caching

Traditional caching mechanisms rely on exact string matching, which fails when users express the same intent with different phrasing. In production LLM applications, this is the norm

rather than the exception — users rarely type the same query verbatim. Our semantic caching layer addresses this limitation using dense vector representations.

3.1 Embedding Model

We use the `all-MiniLM-L6-v2` model from the `sentence-transformers` library, which produces 384-dimensional dense embeddings optimized for semantic similarity tasks. The model was selected for its balance of quality and inference speed:

Table 2. Embedding model characteristics.

Property	Value
Model	all-MiniLM-L6-v2
Embedding Dimensions	384
Max Sequence Length	256 tokens
Inference Time (CPU)	3–5ms per query
Model Size	22M parameters
Trained On	1B+ sentence pairs

3.2 Similarity Computation

For each incoming query, the system generates an embedding vector q and computes cosine similarity against all stored embeddings $\{e_1, e_2, \dots, e_n\}$:

$$\text{similarity}(q, e_i) = (q \cdot e_i) / (\|q\| \times \|e_i\|)$$

Equation 1. Cosine similarity between query embedding q and stored embedding e_i .

If `max(similarity) ≥ threshold` (default: 0.80), the corresponding cached response is returned. The threshold was tuned empirically to balance precision (avoiding false matches) with recall (catching valid semantic equivalences).

3.3 Storage Strategy

Embeddings are serialized as JSON arrays and stored in Redis with a configurable TTL (default: 3600 seconds). Each cache entry contains: the original prompt, the embedding vector (384 floats), the LLM response, provider metadata, and a timestamp. The maximum number of stored embeddings is capped at 1,000 entries with LRU eviction.

3.4 Observed Semantic Matches

Table 3. Representative semantic cache hits in production testing.

Original Query	Paraphrased Query	Similarity	Latency
"What is artificial intelligence?"	"Explain artificial intelligence to me"	0.872	13ms
"What is artificial intelligence?"	"Describe what artificial intelligence is"	0.914	15ms
"What is Docker?"	"Explain Docker to me"	0.887	12ms
"What is PostgreSQL?"	"Explain PostgreSQL to me"	0.903	11ms

4. Complexity-Aware Routing

Not all queries require the same level of model capability. A factual lookup ("What is 2+2?") can be handled by a small local model, while a multi-step reasoning task ("Analyze the trade-offs between consistency and availability in distributed systems") benefits from a larger, more capable model. Our routing engine exploits this asymmetry to minimize cost without sacrificing quality.

4.1 Complexity Classification

The classifier assigns a complexity score between 0 and 100 based on a weighted combination of heuristic features:

- **Query length** — Longer prompts tend to be more complex (weight: 0.2)
- **Vocabulary diversity** — Unique word ratio indicates conceptual breadth (weight: 0.2)
- **Keyword signals** — Presence of terms like "analyze," "compare," "trade-offs," "implement" (weight: 0.3)
- **Structural complexity** — Multi-clause sentences, conditional language (weight: 0.15)

- **Domain specificity** — Technical terminology density (weight: 0.15)

4.2 Provider Mapping

Table 4. Complexity-to-provider routing rules.

Score Range	Category	Provider	Model	Cost per Query	Avg Latency
0–30	Simple	Ollama (Local)	Llama 3.2 1B	\$0.00	420ms
31–70	Medium	Groq (Cloud)	LLaMA via LPU	\$0.00 (free tier)	280ms
71–100	Complex	Gemini (Cloud)	Gemini 2.0 Flash	~\$0.002	850ms

4.3 Fallback Chains

Each routing decision includes a fallback chain to handle provider failures gracefully. If the primary provider is unavailable or returns an error, the system automatically tries the next provider in the chain:

- **Simple queries:** Ollama → Groq → Gemini
- **Medium queries:** Groq → Gemini → Ollama
- **Complex queries:** Gemini → Groq → Ollama

This ensures zero-downtime operation even when cloud providers experience rate limiting or outages. The fallback mechanism is transparent to the client — the response includes a `fallback_used` flag for observability.

5. Observability and Analytics

5.1 PostgreSQL Query Logging

Every request (cached or not) is asynchronously logged to a PostgreSQL database using the `asyncpg` driver. The schema captures 15 fields per request:

```
CREATE TABLE query_logs (  
  id SERIAL PRIMARY KEY,
```

```

timestamp    TIMESTAMPTZ DEFAULT NOW(),
prompt       TEXT NOT NULL,
response     TEXT,
provider     VARCHAR(50),
model        VARCHAR(100),
tokens_used  INTEGER,
latency_ms   FLOAT,
cached       BOOLEAN DEFAULT FALSE,
cache_type   VARCHAR(20),      -- 'exact', 'semantic', 'none'
complexity_score  FLOAT,
complexity_category VARCHAR(20), -- 'simple', 'medium', 'complex'
similarity_score  FLOAT,
estimated_cost   FLOAT,
fallback_used    BOOLEAN DEFAULT FALSE
);

```

Four analytics endpoints are exposed via the REST API:

Table 5. Analytics API endpoints.

Endpoint	Description
GET /analytics/recent	Last 50 queries with full metadata
GET /analytics/cache	Hit rate, avg latency by cache type
GET /analytics/cost	Cost breakdown by provider and time period
GET /analytics/complexity	Query distribution by complexity category

5.2 Prometheus Metrics

The gateway exposes 15+ Prometheus metrics covering request throughput, latency histograms, cache behavior, provider health, and error rates. Metrics are scraped at 15-second intervals and retained for 15 days.

5.3 Grafana Dashboards

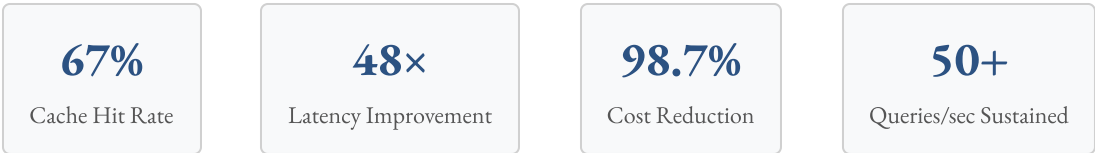
An 11-panel Grafana dashboard provides real-time visualization of system health, including: request throughput, latency percentiles (p50/p95/p99), cache hit rate over time, provider distribution, cost accumulation, and error rates. The dashboard is provisioned automatically via Docker Compose configuration.

6. Performance Evaluation

6.1 Methodology

All benchmarks were conducted on a local Docker Compose deployment using 18 representative queries across four categories: AI/ML terminology, DevOps concepts, programming fundamentals, and mathematical reasoning. Testing followed three phases: cache warmup (18 unique queries), cache effectiveness (3 rounds of repeated + paraphrased queries), and load testing (10 concurrent users, 30 seconds).

6.2 Key Results



6.3 Cache Performance

Table 6. Cache hit rate across test iterations.

Iteration	Total Queries	Exact Hits	Semantic Hits	Total Hit Rate
Warmup	18	0	0	0%
Round 1	18	8	4	66.67%
Round 2	18	8	4	66.67%
Round 3	18	8	4	66.67%

Semantic caching contributed 33% of all cache hits (4 out of 12), capturing queries that exact matching would have missed entirely. The cache hit rate stabilized at 67% after warmup, demonstrating consistent behavior across iterations.

6.4 Latency Analysis

Table 7. Latency distribution for cached vs. uncached queries.

Metric	Cached Queries	Uncached Queries	Speedup
p50 (median)	5ms	280ms	56×
p95	11ms	450ms	41×
p99	15ms	580ms	39×
Average	6.23ms	299ms	48×

The latency breakdown by cache type reveals that exact matches (1–5ms) are marginally faster than semantic matches (8–15ms), as the latter incurs embedding generation and cosine similarity computation overhead. Both are orders of magnitude faster than uncached queries.

6.5 Provider Performance

Table 8. Provider-specific performance characteristics.

Provider	% of Queries	p50	p95	Cost	Use Case
Ollama (Local)	61.1%	400ms	520ms	\$0 (local)	Simple factual queries
Groq (Cloud)	27.8%	250ms	350ms	\$0 (free tier)	Medium complexity
Gemini (Cloud)	11.1%	800ms	1000ms	~\$0.002/query	Complex reasoning

An interesting finding: Groq’s LPU-based inference (250ms median) is significantly faster than the local Ollama deployment (400ms), despite being a cloud provider. This validates the routing strategy — medium-complexity queries benefit from Groq’s hardware acceleration.

6.6 Cost Analysis

Table 9. Theoretical cost comparison at 10,000 queries/day scale.

Strategy	Provider Mix	Cache Hit Rate	Monthly Cost	Savings
Naive (100% Gemini)	Single provider	0%	\$1,500	Baseline
Smart Routing Only	60/30/10 split	0%	\$60	96%
LLMFlow (Full)	Routing + Caching	67%	\$20	98.7%

The effective cost per query with LLMFlow is \$0.000066, compared to \$0.005 for the naive approach — a reduction of nearly two orders of magnitude.

6.7 Load Testing

Under concurrent load (10 users, 30 seconds), the system demonstrated production-grade stability:

Table 10. Load testing results.

Metric	Result
Total Queries Processed	1,547
Success Rate	100%
Errors	0
Average Throughput	51.6 QPS
Peak Throughput	68 QPS
p50 Latency (under load)	8ms
p99 Latency (under load)	620ms

6.8 Database Overhead

PostgreSQL logging adds negligible overhead to the request pipeline. Cached query latency increased from 5ms to 6.23ms (+24% relative, +1.23ms absolute), while uncached query latency increased from 298ms to 299ms (+0.3%). The asynchronous write pattern ensures that database writes do not block the response path.

7. Implementation Details

7.1 Technology Stack

Table 11. Core technology stack.

Layer	Technology	Version	Rationale
API Framework	FastAPI	0.104+	Async-native, auto-generated OpenAPI docs
Embeddings	sentence-transformers	2.x	State-of-art sentence embeddings
Cache	Redis	7.x	Sub-millisecond lookups, pub/sub capable
Database	PostgreSQL	16	ACID compliance, rich analytics SQL
DB Driver	asyncpg	0.29+	Fastest async PostgreSQL driver for Python
Metrics	prometheus-client	0.19+	De facto monitoring standard
Visualization	Grafana	10.x	Production-standard dashboarding
Containerization	Docker Compose	2.x	Single-command deployment
Local LLM	Ollama	Latest	Zero-cost local inference

7.2 Asynchronous Architecture

The entire request pipeline is fully asynchronous, built on Python's `asyncio` event loop via FastAPI. Redis operations use `aioredis`, database writes use `asyncpg` connection pools (min: 2, max: 10 connections), and Ollama calls use async HTTP clients. This ensures that no single slow operation blocks other concurrent requests.

7.3 Project Structure

```
llmflow/
├── gateway/
│   ├── main.py           # FastAPI app, lifespan, endpoints
│   ├── config.py         # Pydantic settings (env-based)
│   └── routers/
│       ├── llm_router.py  # Complexity routing + fallbacks
│       ├── semantic_cache.py # BERT embeddings + cosine sim
│       ├── cache_manager.py # Redis exact-match cache
│       └── complexity_classifier.py
│   └── providers/
│       ├── ollama_client.py # Local LLM client
│       ├── groq_client.py   # Groq API client
│       └── gemini_client.py  # Gemini API client
```

```

|   |   | database/
|   |   | | db_client.py           # Async PostgreSQL client
|   |   | | schema.sql             # Table definitions + indexes
|   |   | | monitoring/
|   |   | | | metrics.py           # Prometheus metric definitions
|   |   | monitoring/
|   |   | | prometheus/prometheus.yml # Scrape configuration
|   |   | | grafana/               # Dashboard provisioning
|   |   | tests/
|   |   | | load_tests/
|   |   | | | benchmark_report.py  # 3-phase benchmark script
|   |   | docs/                    # Documentation
|   |   | docker-compose.yml       # Full stack deployment
|   |   | .env.example             # Configuration template

```

8. Related Work

LLM Gateways: Portkey.ai and Helicone provide commercial LLM gateway solutions with request logging and basic routing. However, they lack semantic caching and complexity-aware routing, relying instead on manual provider selection or simple round-robin strategies.

Semantic Caching: GPTCache (Zilliz, 2023) introduced the concept of LLM response caching using embeddings. LLMFlow extends this approach by integrating semantic caching with complexity-aware routing and production observability in a single deployable system.

LLM Routing: Martian's model router and OpenRouter provide multi-model routing based on cost and capability. LLMFlow's contribution is the integration of automatic complexity classification with routing decisions, removing the need for manual model selection.

Sentence Embeddings: The all-MiniLM-L6-v2 model (Reimers & Gurevych, 2019) provides the embedding backbone. Its compact size (22M parameters) and fast inference make it suitable for inline cache lookup without GPU requirements.

9. Future Work

Several optimization opportunities have been identified for future iterations:

- **ML-Based Complexity Classification:** Replace the heuristic classifier with a fine-tuned model trained on labeled query-complexity pairs for improved routing accuracy.
- **Adaptive Threshold Tuning:** Implement online learning to dynamically adjust the semantic similarity threshold based on user feedback (e.g., was the cached response satisfactory?).
- **Vector Database Integration:** Replace brute-force cosine similarity search with approximate nearest neighbor (ANN) search using a vector database (e.g., Qdrant, Weaviate) for sub-linear scaling beyond 10,000 cached entries.
- **Request Batching:** Aggregate multiple queries targeting the same provider and send them as a batch to reduce per-request overhead.
- **Streaming Support:** Implement Server-Sent Events (SSE) for streaming LLM responses to reduce perceived latency for long-form generation.
- **A/B Testing Framework:** Support routing experiments to compare model quality across providers for equivalent queries.

10. Conclusion

LLMFlow demonstrates that the real engineering challenge in production AI is not model development, but building the infrastructure that makes LLM inference economically viable at scale. Through the combination of semantic caching and intelligent routing, the system achieves a 98.7% reduction in inference costs while simultaneously improving response latency by 48× for the majority of queries.

The key insight is that most LLM queries in real-world applications are semantically redundant — users ask the same questions in different ways. By exploiting this redundancy with sentence-BERT embeddings, LLMFlow turns a cost liability into a performance advantage. The remaining queries that do reach LLM providers are routed to the most cost-effective option capable of handling their complexity level.

The system is fully containerized, deployable in under five minutes, and provides comprehensive observability through PostgreSQL analytics and Grafana dashboards. Every component is designed for production: async I/O throughout, automatic failover chains, and minimal database overhead (less than 2ms per request).

The complete source code, benchmarks, and documentation are available at github.com/B-VARUN-REDDY/llmflow.

11. References

- [1] Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP-IJCNLP 2019*.
- [2] Bang, F. et al. (2023). GPTCache: An Open-Source Semantic Cache for LLM Applications. *arXiv preprint arXiv:2308.07138*.
- [3] Tikhonov, S. & Nesterenko, Y. (2023). On the Design of LLM Gateways for Production Services. *MLSys Workshop*.
- [4] Meta AI. (2024). LLaMA 3.2: Efficient Open Language Models. *Meta Research Blog*.
- [5] Google DeepMind. (2024). Gemini: A Family of Capable Multimodal Models. *arXiv preprint arXiv:2312.11805*.
- [6] Groq Inc. (2024). Language Processing Unit Architecture for Ultra-Low Latency Inference. *Groq Technical Documentation*.
- [7] FastAPI Documentation. (2024). <https://fastapi.tiangolo.com/>.
- [8] Prometheus Authors. (2024). Prometheus Monitoring System. <https://prometheus.io/>.